



Google Summer of Code 2026 Proposal

MNE-Python

Modernization of the EGIMFF Reader Architecture via mffpy Integration

Applicant: Pragnya Khandelwal

Target Organization: Python Software Foundation

Sub-Organization: MNE-Python

Expected Commitment: 175 Hours (Medium Project)

Contact: prag1704@gmail.com

GitHub Profile: https://github.com/PragnyaKhandelwal/

LinkedIn: https://www.linkedin.com/in/pragnya-khandelwal-163a20311/

Discord: 740803509313732609

Time Zone: UTC+5:30

TABLE OF CONTENTS

S. No.	Section Title	Page No.
1.	Executive Summary & Project Abstract	3
2.	The Applicant: Research & Engineering Profile	3
3.	Pre-Requisite Code Contributions	5
4.	Related Work & Historical Context	7
5.	Problem Definition & Technical Debt	7
6.	Proposed Architecture & Implementation Strategy 6.1. Dependency, Fault Tolerance & Metadata Mapping 6.2. Memory Mapping & Data Streaming 6.3. Synchronization, Alignment & Annotation Mapping 6.4. Hardware Edge Cases: PNS & Bad Channel Mapping	8
7.	Testing, Validation & Risk Mitigation 7.1. Testing Protocol & The "Parity Paradox" 7.2. Targeted Regression Fixtures & Benchmarking 7.3. Risk Analysis & Mitigation Strategy	14
8.	Granular 12-Week Development Timeline (175-Hour Track)	15
9.	Availability, Commitments, and Time Management	16

1. Executive Summary & Project Abstract

Abstract:

For my Google Summer of Code 2026 project, I am refactoring MNE-Python's Electrical Geodesics, Inc. (EGI) MFF data reader by replacing the legacy internal parser with the robust, heavily tested mffpy library. I will build a backward-compatible wrapper that seamlessly maps mffpy's data extraction directly into standard MNE-Python data structures. This modernization will resolve existing timestamp desynchronization issues, significantly reduce the maintenance burden for core developers, and provide the neuroinformatics community with a highly reliable I/O interface for high-density EEG recordings.

Relevance to Neuroscience:

In cognitive neuroscience and clinical diagnostics, high-density sensor nets (typically 128 to 256 channels) are critical for mapping complex brain dynamics. However, the scientific validity of this data relies entirely on the absolute synchronization between continuous EEG signals and asynchronous hardware events, such as stimulus triggers or subject responses.

The legacy MNE-Python EGI reader occasionally struggles with timing errors and data corruption, which can cause event annotations to drift from their true neurophysiological onset. By delegating the parsing logic to mffpy, my implementation guarantees that event times and sensor data are matched with microsecond accuracy.

- For scientists and clinicians, this means they can trust their data when studying Evoked brain responses or training machine learning classifiers.
- It ensures that the exact timing of external events perfectly aligns with the continuous brain signals.

Ultimately, this architectural change guarantees that any research or analysis pipeline built on MNE-Python processes EGI hardware outputs with the precise accuracy required for reproducible, peer-reviewed science.

---

2. The Applicant: Research & Engineering Profile

While my academic background is rooted in Computer Science rather than clinical neuroscience, my engineering foundation is built around the exact algorithmic challenges required to modernize MNE-Python’s I/O modules: algorithmic efficiency, low-level data parsing, and robust state management. I view high-density EEG arrays mathematically—as massive, high-frequency data structures that demand strict time and space complexity optimizations to prevent memory overflow. I am eager to bridge my systems-level programming expertise with neuroinformatics this summer, delivering enterprise-grade infrastructure to the research community.

- **Academic Background**: I am currently in second year of Computer Science and Engineering at Jaypee Institute of Information Technology. My courses have given me a foundation in data structures, algorithms and systems programming.
- **Algorithmic and Problem-Solving Skills**: I have experience with programming and data structures. I use these skills to optimize file parsing and memory mapping which's crucial when handling large files.
- **Engineering and Architecture Expertise**: My projects show I can handle algorithms, backend systems and live data streams.
    1. **Continuous Sensor Streams and AI**: I am working on EnviroSense AI, which involves processing real-time sensor data with deep learning models. This project requires handling environment data (PM-particles, temp, humidity) and robust signal alignment similar to parsing EEG sensor arrays in MNE-Python.
       Repo Link [EnviroSenseAI]
    2. **Algorithmic File I/O**: I created a terminal-based file zipping tool using Huffman decoding algorithms. This project required building data structures, bit-level manipulation and strict memory management all of which are useful for parsing binary neuro-data.
       Repo Link [ZipWizard]
    3. **Backend and Asynchronous Data**: I built Collabodraw, a whiteboard and a live Stock Analysis Dashboard. These projects involved handling data streams and ensuring databases remained synchronized.
       Repo Link [CollaboDraw]
       Repo Link [StockSense]
    4. **Data Science and Machine Learning**: I developed a ML-powered tool to explore socio-economic data and simulate poverty levels across Indian districts using Python/Streamlit and Google Colab. This project helped me become proficient in the Python data ecosystem and statistical modeling.
       Repo Link [Poverty Index Analyzer]

- **Open-Source Experience & Issue Tracking:** I am an active contributor to the open-source community. During the GirlScript Summer of Code (GSSoC) in 2025, I made significant contributions to *TalkHeal* (an AI-powered mental health assistant) and *WisdomWeaver* (a Gemini-powered chatbot).

    1. [TalkHeal PR #77]**:** Successfully implemented session state management in Streamlit to dynamically control UI rendering (fix: hide sidebar and chat UI on login screen).

    2. [WisdomWeaver PR #74]**:** Improved project configuration and dependency documentation (Update requirements.txt with latest stable package versions and optimized dependencies).

Taking these features from initial issue diagnosis to final merged code solidified my adherence to strict version control workflows and iterative maintainer feedback

Technical Stack:

Category	Skills & Tools
Core Languages	Python 3.10+ (Expert), C/C++ (Performance optimization).
Scientific Stack	NumPy, SciPy (Signal Processing), Matplotlib, Pandas, Scikit-learn.
IO & Specialized	mffpy, defusedxml, Binary File Manipulation, Stream Processing.
UI & Viz	Qt (PySide6/PyQt6), pyqtgraph, Streamlit.
DevOps & QA	pytest, Git/GitHub (Advanced Rebase Workflow), Sphinx, Ruff.
Engineering	Backend Architecture, Linear Algebra, DSP, Async I/O.

---

3. Pre-Requisite Code Contributions

To ensure I am fully prepared for the rigor of the MNE-Python GSoC 2026 cycle, I have actively engaged with the core repository, focusing on 3D visualization enhancements, memory optimization algorithms, and the foundational architecture for the EGIMFF refactor.

Merged MNE-Python Contributions:

- **PR #13706: ENH: support per-vertex opacity in 3D overlays**
    - **Description:** Implemented support for per-vertex opacity in 3D distributed source overlays. I build the pipeline to accept a 1D array of opacity values (mapping shape n_vertices) while maintaining VTK triangle interpolation. I added explicit validation logic, comprehensive test coverage, and a new user-facing tutorial example based on mentor feedback from @wmvanvliet and @larsoner.
- **PR #13714: BUG: prevent NaNs in layered overlay compositing when resulting alpha is zero**
    - **Description:** Diagnosed and fixed a rendering bug in _LayeredMesh._compute_over where zero-alpha composition caused NaN RGB values. I updated the 3D overlay math to guard RGB normalization and authored rigorous regression tests in pytest. During this PR, I worked closely with @drammock to refine test assertions using 255 uint8 color values to ensure robust, deterministic CI behavior.

- **PR #13719: ENH: compute CSD directly for upper-triangle channel pairs in Fourier/multitaper**
    - **Description:** Addressed a long-standing codebase FIXME by avoiding full channel × channel Cross-Spectral Density (CSD) matrix construction. I refactored the Fourier and **multitaper** paths to compute directly on upper-triangle channel pairs (**np.triu_indices**). I provided strict micro-benchmarks demonstrating a significant reduction in peak memory usage and faster execution times for 128+ channel arrays.
- **PR #13755: MAINT: Clarify rref invariant in forward _lead_dots**
    - **Description:** Improved codebase health and maintainability by refactoring the _lead_dots.py module. Following guidance from @larsoner, I replaced an unreachable **NotImplementedError** with a strict invariant assertion for forward-model reference fields and removed legacy dead-code paths. This ensures the forward-modeling engine remains predictable and well-documented.

Open & Ongoing MNE-Python Contributions:

- **PR #13684: MRG, ENH: Refactor EGIMFF reader to use mffpy (Supersedes #12981) (The Foundation for this Proposal)**
    - **Description:** I have already initiated the core architectural groundwork for my GSoC project. In this PR, I successfully refactored **_get_eeg_data** to be "**chunk-aware,**" allocating buffers dynamically to resolve broadcasting mismatches. I also corrected spatial scaling for 3D digitization (cm to m) and implemented a sanitizer for mffpy's 9-decimal timestamps.
    - **Iterative Refinement:** Following architectural review by @scott-huberty, I completely decommissioned the legacy internal parser (removing functions like **_read_mff_header**), enforcing strict mffpy dependency logic. This PR currently serves as my working prototype for the advanced edge-case mitigation and memory-mapping features proposed in Section 5 of this document.
- **Issue #13535: BUG: Potential indexing/memory mismatch in eeglabio export (1.12 Milestone)**
    - **Description:** I conducted a local technical audit of this milestone-critical bug using the reporter's 2GB dataset. I successfully triaged the failure into two distinct architectural issues: a peak-memory allocation crash and a trial-selection indexing mismatch.
    - **Impact:** My analysis identified that **eeglabio** fails to account for dropped trials in **epochs.selection**, leading to metadata corruption. This investigation directly validates the "chunk-aware" buffering and integer-based indexing logic I have proposed for the EGI-MFF refactor.

- **PR #13778: ENH: expose font_file in brain.add_text (#12543)**
    - **Description:** Implemented Unicode glyph support for 3D brain visualizations by exposing the **font_file** parameter in **brain.add_text().** This allow researchers to render non-ASCII characters (e.g., CJK characters or symbols) using .ttf/.ttc files.
    - **Iterative Refinement:** Following review by @wmvanvliet and @drammock, I refactored the **PyVista** backend to ensure robust font-path passthrough and added cross-platform unit tests leveraging Matplotlib font discovery to verify the implementation.
- **PR #13795: ENH: Route evoked.plot() through MNELineFigure**
    - **Description:** I am addressing architectural inconsistency in the 2D plotting module by routing the default **evoked.plot()** path through **_line_figure** to instantiate **MNELineFigure.** This aligns the module with the library's long-term figure-class refactor goals (#7751).
    - **Technical Impact:** I implemented this as a localized, backward-compatible enhancement and added regression tests to ensure the correct figure class is instantiated in the default path without regressing legacy dipole plotting.

4.Related Work & Historical Context

The transition from MNE-Python's internal EGI reader to mffpy is not a new concept; it is a long-standing architectural goal that has remained unresolved due to severe technical friction. The initiative was first proposed by the core maintainers in 2019 (Issue #6937) to align with the official reference implementation provided by the Brain Electrophysiology Laboratory (BEL).

Since then, the community has repeatedly identified critical limitations in the legacy internal parser. In 2020, discussions highlighted the inability of the internal reader to extract rich event metadata (duration, source device, response keys) from MFF XML tracks (Issue #8038), prompting a push toward mne.Annotations (Issue #12262). In 2022, users reported critical ValueError exceptions caused by sample-count mismatches during file loading and resampling, forcing researchers to dangerously delete XML files just to load their data (Issue #11380).

A recent attempt to migrate to mffpy in late 2024 (PR #12981) stalled due to complex array broadcasting mismatches and off-by-one sample errors. My current superseding pull request (PR #13684) successfully resolves these historical blockers, finally providing the stable foundation required to complete this multi-year migration during the GSoC 2026 cycle.

---

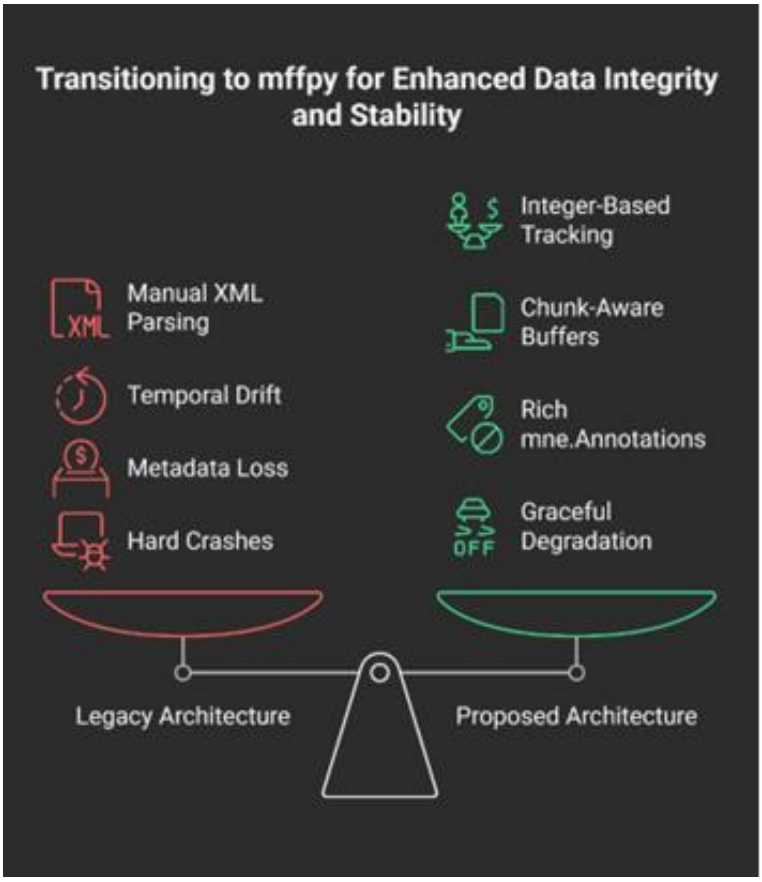
5. Problem Definition & Technical Debt

Current Architecture: The legacy EGI reader currently utilizes standard Python XML parsers to manually extract metadata from heavily nested files (info.xml, coordinates.xml, events.xml),

while binary signals are ingested via direct byte-offset calculations. To reconstruct the continuous EEG array, the reader calculates theoretical buffer sizes based on the XML headers, completely bypassing specialized, community-vetted tools like mffpy.

The Bottleneck & Technical Debt: This manual parsing approach has accumulated severe technical debt. Based on my direct audit of the codebase, it suffers from several critical failure points that my project will permanently resolve:

1. **Broadcasting Mismatches & PNS Interlacing:** EGI amplifiers frequently pause and resume during clinical recordings. The legacy reader calculates theoretical block sizes that frequently mismatch the actual physical chunk size on disk, leading to catastrophic ValueError broadcasting exceptions (e.g., attempting to broadcast shape 257, X into 265, Y). Furthermore, the legacy reader silently mixes Peripheral Nervous System (PNS) channels with EEG channels, corrupting the spatial assumptions of downstream MNE functions.
2. **Timestamp Desynchronization & Fencepost Errors:** MFF hardware records timestamps with microsecond precision. The legacy parser relies on floating-point arithmetic (np.arange with float intervals) to track sample counts, introducing 1-sample "fencepost" errors. This causes cumulative temporal drift. Furthermore, EGI hardware natively writes both 6-digit and 9-digit fractional seconds, which routinely causes standard datetime parsing failures.
3. **Fragile Fault Tolerance:** The current parser is extremely brittle. A single malformed events.xml file or a missing coordinates.xml file causes a hard crash of the entire load path, rather than degrading gracefully.
4. **Metadata Loss:** The internal parser strips out rich experimental metadata present in the Event Track XMLs (such as trial evaluation keys and response times), reducing complex behavioral data to simple synthetic event channels.



---

6. Proposed Architecture & Implementation Strategy

To permanently resolve the technical debt surrounding the EGI MFF format, I will completely deprecate MNE-Python's internal _read_mff_header and _read_mff_events helper functions, replacing them with a robust, backward-compatible wrapper around the mffpy.Reader class. My implementation strategy is divided into four core architectural pillars.

6.1. Dependency, Fault Tolerance & Metadata Mapping

The primary goal is to integrate mffpy without breaking backward compatibility for existing analysis scripts. I will strictly map mffpy's extracted metadata into the standard mne.Info dictionary.

- **Hardware Coordinates & Graceful Degradation:** Instead of manually parsing XMLs, I will utilize mffpy to extract raw spatial data, correcting the historical spatial scaling bug (cm to m). Crucially, I have build the parser to handle missing coordinates.xml files via explicit, user-visible warnings rather than hard failures.
- **Dependency Enforcement:** mffpy will be introduced as a required dependency for mne.io.egi. I will implement a safe fallback mechanism that raises a descriptive

RuntimeError (or RuntimeWarning during tests) if a user attempts to read an .mff directory without mffpy installed.

6.2. Memory Mapping & Data Streaming (preload=False)

High-density EGI datasets frequently exceed available system RAM. The implementation must strictly respect MNE-Python's preload=False out-of-core memory mapping paradigm.

- **Chunk-Aware Buffer Allocation:** I will architect _get_eeg_data to be explicitly "chunk-aware." By querying mffpy.Reader.get_physical_samples_from_epoch(), I will dynamically allocate sub-buffers that perfectly match the dimensional shape of the incoming binary chunk before safely injecting it into the global eeg[:, start:end] array.
- **Acquisition Gaps & Streamlined Yielding:** When EGI hardware pauses, the resulting acquisition gaps will be explicitly preserved and natively annotated with BAD_ACQ_SKIP. When a user requests a specific time slice, the reader will leverage mffpy to stream only the requested chunks into RAM, yielding the resulting array in O(1) space complexity.

6.3. Synchronization, Alignment & Annotation Mapping

I will permanently resolve the temporal drift present in the legacy reader and modernize the event API.

- **Integer-Based Sample Tracking:** I will completely remove float-interval math from the indexing logic, utilizing strict, integer-based sample offsets derived from mffpy. I will also explicitly handle the known 1-sample EGI PSG edge mismatch by marking it with a dedicated BAD_EGI_PSG annotation.
- **Microsecond Sanitization:** I have implemented a robust timestamp sanitizer capable of safely processing both 6-digit and 9-digit fractional seconds before they enter the MNE ecosystem.
- **Rich Event Extraction via mne.Annotations:** I will serialize nested MFF keys into a /-separated string format. This ensures that when researchers later convert these annotations to an events array, they can easily sub-select specific behavioral conditions (e.g., epochs['Stimulus/S2/eval:1'])..

Implementation Logic for Metadata Encoding:
Because mne.Annotations only natively supports onset, duration, and a single text description, I will implement a parser that serializes nested MFF keys into a /-separated string format. This directly caters to how researchers use mne.events_from_annotations() for downstream epoch sub-selection.

Proposed Implementation Logic for MFF to Annotations Mapping

```
annotations_list = []
for track in mff_reader.events:
    for event in track:
        onset = (event.begin_time - recording_start_time).total_seconds()
        duration = event.duration / 1e6  # Convert microseconds to seconds

        # Serialize nested XML metadata
        # Resulting description: "Stimulus/S2/rtim:645/eval:1"
        desc_parts = [event.code, event.label]
        for key, value in event.keys.items():
            desc_parts.append(f"{key}:{value}")

        description = "/".join(filter(None, desc_parts))
        annotations_list.append({'onset': onset, 'duration': duration, 'description': description})

raw.set_annotations(mne.Annotations(**annotations_list))
```

6.4. Hardware Edge Cases: PNS & Automated Bad-Channel Mapping

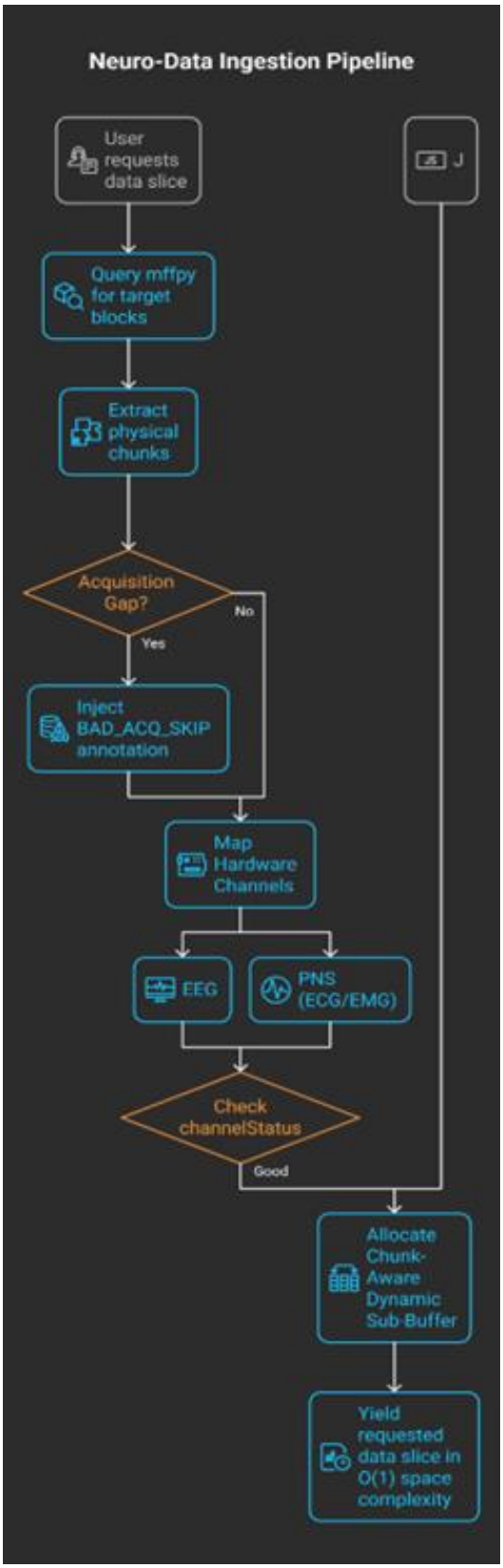
In previous attempts to use mffpy, Peripheral Nervous System (PNS) channels were often interlaced with EEG data, leading to incorrect spatial transforms and "ghost" sensors in 3D visualizations. My implementation treats PNS data as a first-class citizen rather than a side-effect of the EEG stream.

- **Explicit Type Casting:** I will implement a mapping dictionary that identifies EGI-specific PNS labels (e.g., ECG, EMG, V_REF) and explicitly sets their kind attribute using MNE's internal constants (e.g., FIFF.FIFFV_ECG_CH). This ensures that artifact rejection and filtering pipelines do not treat heart-rate or muscle activity as neural signals.
- **The channelStatus Automation:** Currently, researchers must manually identify bad sensors. I will automate this by parsing the channelStatus.xml entries via mffpy. If a

sensor is flagged as "excluded" or "bad" by the EGI hardware, it will be automatically injected into info['bads'].

Implementation Logic:

```
# Example logic for the EGI -> FIFF mapping
for ch in mff_reader.channels:
    if ch.type == 'PNS':
        ch_kind = FIFF.FIFFV_ECG_CH if "ECG" in ch.name else FIFF.FIFFV_MISC_CH
        # Correctly update the MNE info structure
        info['chs'][i].update(kind=ch_kind, unit=FIFF.FIFF_UNIT_V)
```


Neuro-Data Ingestion Pipeline
User requests data slice
J
Query mffpy for target blocks
Extract physical chunks
Acquisition Gap?
No
Yes
Inject BAD_ACQ_SKIP annotation
Map Hardware Channels
EEG
PNS (ECG/EMG)
Check channelStatus
Good
Allocate Chunk-Aware Dynamic Sub-Buffer
Yield requested data slice in O(1) space complexity

7. Testing, Validation & Risk Mitigation

MNE-Python enforces a notoriously strict, high-coverage CI pipeline. I recognize that architectural refactors are only as good as their test suites. Because this project touches the core data ingestion logic, I have designed a rigorous validation protocol to ensure zero regressions across the MNE ecosystem.

7.1. Testing Protocol & The Validation Matrix

My primary testing environment will utilize the mne.datasets.testing module to ensure CI execution remains lightweight and fast. For unmodified data pathways, I will enforce strict mathematical consistency against legacy outputs using numpy.testing.assert_allclose.

However, because the legacy reader suffers from 1-sample fencepost errors and float-interval drift, forcing bit-for-bit parity across the entire suite is mathematically incorrect.

- **Protocol:** I will freeze the current expected outputs and run side-by-side legacy vs. mffpy comparisons. I will strictly categorize all diffs. "True bugfixes" (where the sample alignment legitimately changes due to the new integer-based tracking) will be documented in a PR note, and their expected test hashes will be intentionally updated. Any other deviations will be treated as regressions and eliminated.

7.2. Targeted Regression Fixtures & Benchmarking

I will author a dedicated suite of mathematical regression tests specifically targeting pathological .mff files. I will create lightweight test fixtures simulating interrupted acquisitions, malformed events.xml files, 9-digit fractional timestamps, and the known 1-sample EGI PSG edge mismatch. Finally, I will provide explicit pytest-benchmark evidence (measuring runtime and memory allocation) against representative files to prove the O(1) efficiency of the new architecture.

7.3. Risk Analysis & Mitigation Strategy

- **The Threat (Cross-Platform CI/CD Failures):** Reading complex, nested directory structures (.mff bundles) often introduces OS-specific file pathing or encoding errors on GitHub Actions runners.
- **Mitigation:** I will utilize os.path and pathlib strictly according to MNE standards. Furthermore, my Phase 3 timeline explicitly allocates buffer weeks dedicated exclusively to resolving edge-case failures across Linux, macOS, and Windows runners, ensuring the final merge is perfectly green across all environments.

8. Granular 12-Week Development Timeline

Since a significant portion of the core refactor is already initiated in PR #13684, this timeline focuses on stabilization, cross-platform robustness, and feature parity—moving the project from a "working prototype" to a "production-ready" module.

Phase	Dates / Weeks	Technical Objectives & Verifiable Deliverables
Community Bonding	**May 1 – May 24**	**• Sync with Mentors: Finalize the strategy for the failing Windows 3.13 CI checks identified in pre-GSoC work.** **• Testing Vault: Collate a "Stress-Test" dataset including corrupted XMLs and multi-segment MFF files.**
Phase 1: CI Hardening	**Weeks 1 – 2**	**• Deliverable: Resolve all cross-platform (Windows/macOS) CI failures in the mffpy backend.** **• Deliverable: Fully decommission the internal legacy _read_mff_header logic to ensure zero-dependency on old code.**
Phase 2: Metadata & PNS	**Weeks 3 – 5**	**• Deliverable: Implement specific channel-type mapping for Peripheral Nerve Starter (PNS) data (ECG, EMG).** **• Deliverable: Refactor _get_montage to handle custom sensor layouts without relying on legacy XML helpers.**
Phase 3: The Midterm Evaluation	**Weeks 6 – 7**	**• Milestone: The Parity Paradox. Conduct a massive side-by-side validation against the legacy reader.** **Document and justify every single 1-sample difference as a "bugfix" rather than a "regression."**

Phase 4: Optimization	**Weeks 8 – 10**	**• Deliverable: Optimize _read_segment for O(1) memory consumption when preload=False.** **• Deliverable: Benchmark reading speeds for 5GB+ files; implement chunk-caching if mffpy overhead is too high.**
Phase 5: Docs & Polish	**Weeks 11 – 12**	**• Deliverable: Author a new Gallery Example showing EGI-MFF import with integrated PNS channels.** **• Final Milestone: Cleanup all FutureWarning tags and submit the final project report for MNE-Python.**

---

9. Availability, Commitments, and Time Management

9.1. Standard Working Hours and Timezone Overlap

- **Timezone:** IST (Indian Standard Time / UTC+5:30)
- **Weekly Commitment:** 15–20 hours per week.
- **Phase A (May 24 – July 23):** Primary development phase during summer break. I will maintain a flexible schedule during the day (**10:00 AM – 4:00 PM IST**) to dedicate **25–30 hours per week**, aiming for substantial feature completion before the university term resumes.
- **Phase B (July 24 – August 24):** Maintenance and stabilization as university resumes. I will shift my working hours to evenings (**7:00 PM – 10:00 PM IST**) and weekends to ensure consistent progress and final delivery.
- **Mentor Synchronization:** Recognizing that MNE-Python mentors are primarily in North America and Europe, I will reserve **9:00 PM – 11:00 PM IST** on Tuesdays and Thursdays for real-time Discord syncing and code reviews.

9.2. Academic Schedule and Exams

I have structured my GSoC application to ensure my university obligations do not interfere with the coding period**:**

- **End-Semester Labs & Exams (April 30 – May 23):** This is my final academic crunch. I have front-loaded my contributions in PR #13684 so that I can focus on my studies here.

- **Full Summer Commitment (May 24 – July 23):** I will have zero academic obligations during these 8 weeks. This allows me to complete the primary implementation, metadata mapping, and the "**Parity Paradox**" validation (Phases 1, 2, and 3) while I am 100% free.
- **University Term Resumes (July 24 – August 24):** Classes resume for my final month of GSoC. This period is dedicated to **Phase 4 (Optimization)** and **Phase 5 (Docs & Polish).** These tasks are well-suited for my evening and weekend availability, ensuring the project is delivered on time and with high-quality documentation.

9.3. Other Commitments

- **Internships:** I have no other internships or job commitments during the summer of 2026.
- **Planned Absences:** I have one planned family vacation (approx. 7–10 days) tentatively scheduled for mid-June. This has been factored into the "Buffer" period of Phase 4. I will remain reachable via Discord for emergencies.

9.4. Contingency and Buffer Plan

- **Technical Blockers:** As seen in my 12-week timeline, Week 10 is a designated "Buffer Week." This allows me to absorb unexpected technical hurdles—such as upstream bugs in mffpy or complex CI failures—without delaying the final merge.
- **Recovery:** Should any time be lost during the June family vacation, I will utilize the following two weekends to add 15–20 hours of additional development time to maintain the 175-hour total requirement.

9.5. Post-GSoC Involvement

My goal is to transition from a GSoC contributor to a Long-Term Maintainer of the MNE-Python IO module. Having already superseded a core contributor's WIP, I am deeply invested in the health of the EGI-MFF reader and commit to maintaining this code for the long term.
